2023 IEEE Canadian Conference on Electrical and Computer Engineering (CCECE) | 979-8-3503-2397-9/23/$31.00 ©2023 IEEE | DOI: 10.1109/CCECE58730.2023.10288905

An Efficient VHDL Implementation of two Artificial Neural Networks on Zynq-7000 FPGA

Kaveh Sadeghikhah
Engineering and Applied Science
University of Regina
Regina, Canada
ksh902@uregina.ca

Lei Zhang
Engineering and Applied Science
University of Regina
Regina, Canada
lei.zhang@uregina.ca

Raman Paranjape
Engineering and Applied Science
University of Regina
Regina, Canada
raman.paranjape@uregina.ca

***Abstract*—This paper presents the FPGA implementation of two different topologies of an Artificial Neural Network (ANN) on the Xilinx Zynq-7000 evaluation board. The engine dataset available in MATLAB is used to train the neural network. The resulting parameters of a neural network are taken from MATLAB and are used to implement it on FPGA. Two structures are implemented with different accuracy of sfix_24_8 and sfix_32_16 and different clock frequencies and resource utilization is measured. The maximum achievable frequency measured is 83.33 MHz and the minimum power is 0.203 W.**

Keywords— artificial neural network, FPGA, VHDL, training, activation function, MATLAB, weights, and biases.

I. Introduction

The artificial neural network structure in this experiment consists of an input layer, a hidden layer, and an output layer. Artificial neural networks are used for forecasting and classification in fields like medicine, military, and finance due to their flexibility and ability to process different types of data quickly through parallel processing [1]. The simulation is based on the "engine_dataset" provided in MATLAB that outlines the engine behavior. Furthermore, there is a single hidden layer in the neural network architecture. While software-based artificial neural networks (ANNs) can be implemented with ease, one downside is that larger neural networks tend to have slower execution times. As a result, they may not be suitable for many real-time applications [2]. Function fitting refers to training a neural network to produce a set of target outputs based on a set of inputs. Once the network has learned the data, it can generalize the input-output relationship and provide outputs for new inputs that it was not initially trained on. This dataset can be used to train a neural network to estimate an engines torque and emissions from its fuel use and speed. By loading the dataset in MATLAB, we have two variables as follows.

engineInputs which is a 2x1199 matrix defining two attributes of a given engines activity under different conditions named as fuel rate and speed.

engineTargets which is a 2x1199 matrix of two attributes to be estimated given the inputs as torque and nitrous oxide emissions.

II. Training and Simulation

The data division in the training process is random while the training method is relying on the famous Levenberg-Marquardt together with the calculation of Mean Square Error to assess the performance of the neural network. Two different experiments have been simulated by changing the number of neurons in the hidden layer to 4 and 8, respectively. Figure 1 depicts the architecture of the neural network with 2 inputs, 4 hidden layers, and 2 outputs. The dataset includes 1200 samples which are divided into three categories. The train data consists of 70% of the dataset and 15% of the whole dataset belongs equally to the validation and test data. The dataset includes 1199 samples which are divided into three categories. Since the data of the neural network is continuous the MSE metric has been considered for the performance. Moreover, the number of epochs in all three experiments is set to 500.

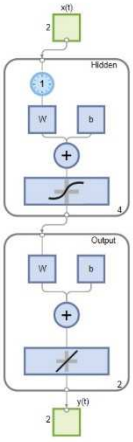


Figure 1. The architecture of a neural network with 2 inputs, 4 hidden layers, and 2 outputs for the engine_dataset

The details of the artificial neural network training process such as epochs, gradient, validation checks, etc. as well as the results including Mean Squared Error (MSE), Root Mean Square Error (RMSE), mean, and standard deviation have been investigated before and presented in Figure 2. The MATLAB simulation is based on floating-point calculations. The default setting for the MATLAB and Simulink blocks, values, and functions is the double floating point which resembles a 64-bit

XXX-X-XXXX-XXXX-X/XX/$XX.00 ©20XX IEEE

Authorized licensed use limited to: Cyprus University of Technology. Downloaded on June 04,2025 at 12:04:18 UTC from IEEE Xplore. Restrictions apply.

length of the data. This consists of the sign, integer, and fraction segments. Since we need to implement the artificial neural network on the FPGA the double precision of the data seems to utilize a lot of hardware resources hence the power consumption increases which is not ideal. Therefore, we should consider a lower amount of bit length for my ANN calculations such that the utilized resource and power consumption will be reasonable to the extent that we will not lose the accuracy. That is where fixed-point representation comes into play and all the data including inputs, coefficients, and arithmetic operations of addition and multiplication must be revised to represent this computation system [3].

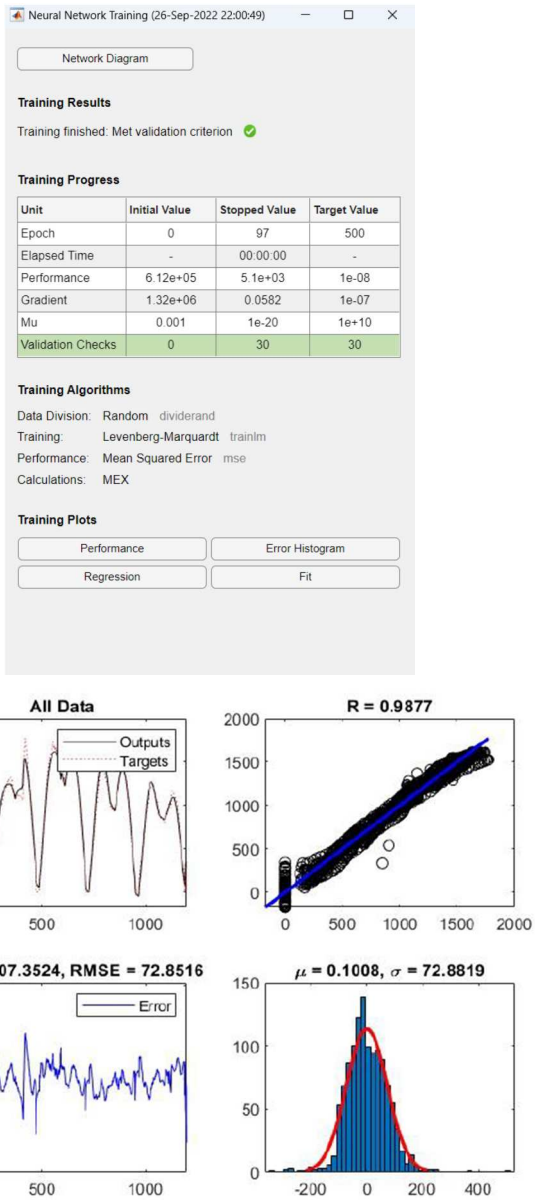


Figure 2. The floating-point model of ann_2_4_2 in Simulink

The inputs are ranging from 0.6 to 1801.8. Therefore, all the calculations in MATLAB are done within the floating-point number system based on these values. In order to ease the calculations of my system to be presented as fixed-point values we better normalize the input values. This normalization process has been done in VHDL to better cope with the hardware implementation outputs and to verify the results in the real hardware of the Zedboard Evaluation Kit. The structure of the ann_2_4_2 floating point model has been shown in the Figure 3. The dimension and type of the signals are double in this figure as there is no quantization tools being used all the data are floating-point.

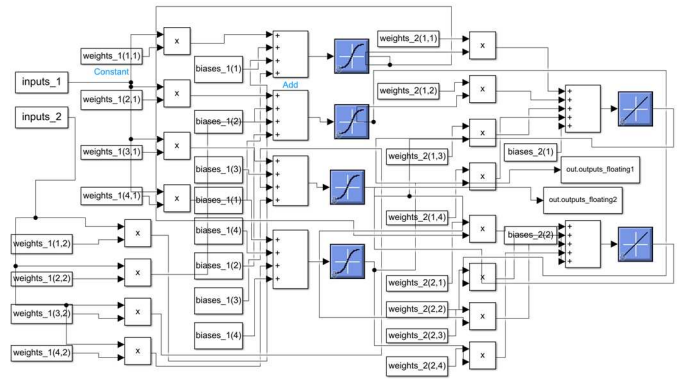


Figure 3. The floating-point model of ann_2_4_2 in Simulink

To make the ready to be used for the VHDL coding we must quantize the values to use the fixed-point numbers. This has been done in Simulink and shown in the Figure 4 [4]. It is noted that the calculations are accomplished using signed numbers and because of the range of the input the fixed-point representation is sfix_18_6. This length will be extended as the input signal propagates through the neural network due to the multiplication and addition with weights and biases [5]. For the addition, the bit length of the results equals the biggest bit length of the operands while for the multiplication case the integer part equals m+n+1 in which m and n are the bit length of two operands for the multiply and the fractional part equals f_1+f_2 which the f_1 and f_2 stand for fractional bit length of the two operands. This causes the extension of the bit while we go through the graph. This leads to the output of sfix_24_8 as visible in Figure 3. The ann_4 and ann_8 both has been analyzed and compared with sfix_24_8 and sfix_32_16 architectures. Therefore, there will be four different topologies to compare. It is to mention that all the bit length of all values such as weights and biases has been assigned depending on their range as a result of MATLAB simulation. The activation functions for the input and hidden layer used in this ANN are "tansig" and "purelin", respectively. For the tansig function, it is required to insert the double data type therefore a conversion prior to and following this block has been done in the fixed-point model.

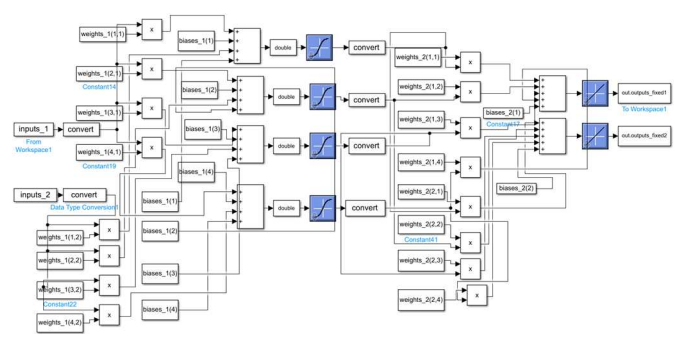


Figure 4. The sfix_24_8 fixed-point model of ann_2_4_2 in Simulink

Authorized licensed use limited to: Cyprus University of Technology. Downloaded on June 04,2025 at 12:04:18 UTC from IEEE Xplore. Restrictions apply.

The purpose is to reduce the bit length such that the outputs will be perfectly aligned with those of floating-point values so that we can fit the artificial neural network in the FPGA. The architecture of the FPGA is a perfect choice for the ANN implementation as it mostly uses adders and multipliers which are abundant in the DSP blocks of the FPGA [6]-[7]. To validate that the accuracy of the fixed-point model its output has been plotted with the floating-point for comparison. This plot has been demonstrated for the sfix_24_8 and sfix_32_16 models in Figure 5. As can been seen in the Figure 5 there is not a significant error when using sfix_24_8 model compared to the sfix_32_16 so the sfix_24_8 model has been chosen as a base model for the design and implementation in FPGA. By having this fixed-point configuration, the aim is to calculate the amount of error generated due to the quantization of different values which is known as quantization error. It is possible to calculate this amount with the help of the mean-square concept that is MSE. The formula of MSE is shown below.

$$\mathrm{MSE} = \frac{1}{n}\sum_{i=1}^{n}(Y_i - \hat{Y}_i)^2$$

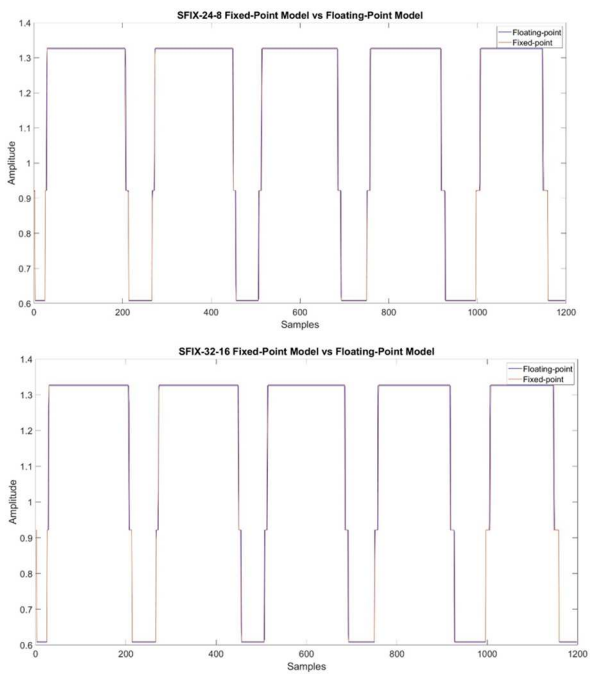


Figure 5. The floating-point and fixed-point comparison: (a) sfix_24_8 (b) sfix_32_16

The resulting MSE errors of the four different structures are listed for the two different outputs in the Table I. As it is apparent increasing the bit length from 24_8 to 32_16 will not improve the performance dramatically thus the main model is set to be sfix_24_8 for the FPGA Implementation.

Table I. MSE Error for Four Different ANN Architectures

MSE Error	ann_4		ann_8	
	sfix_24_8	sfix_32_16	sfix_24_8	sfix_32_16
output1	0.0014	0.0014	0.0346	0.0345
output2	0.0107	0.0109	0.0251	0.0250

III. FPGA Implementation

Having the sfix_24_8 as the fixed-point model now it is time to write the VHDL code for this architecture to implement it on Zedboard Evaluation Board [8]. The top module written is called "ann.vhd" which includes all the submodules required for the implementation of the artificial neural network. To parametrize the design a package has been developed for the artificial neural network named "ann_package.vhd". The innovation of this design is the generic consideration so that easily the neural network can be modified based on the application. Here, all the essential parameters of a neural network have been defined. These include number of inputs, hidden layers, and outputs, data representation constants for the integer and fractional parts, array definition for signals, biases, weights, and normalization parameters. Moreover, the parameters of the trained neural network must be fed into this package including inputs, hidden layer and output layer biases, hidden layer and output layer weights, and normalization parameters. The array definition here makes it possible to input these values in the package.

To optimize the implementation, it is crucial to normalize the input and output values as there is a wide range of them. Two different submodules named "input_processing.vhd" and "output_processing.vhd" have been written for this purpose. To connect the inputs to the hidden layer a submodule named "n_hidden.vhd" has been written. In this code the number of required signals will differ depending on the hidden layer size as for the basic structure is 4. Similarly, to connect the hidden layers to the outputs there is a need for another submodule named "n_output.vhd" in the design. Furthermore, to preserve the nonlinear behavior of the neural network there has to be activation functions in the design just as we did in the MATLAB simulation [9] hence the "tansig" function for the hidden layer and "purelin" function for the output are developed in two distinct submodules named "tansig.vhd" and "purelin.vhd", respectively. The length of the integer and fractional parts must be specified in these two files for accurate performance otherwise it will generate an error when simulating the design. Since there is a need for an array multiplication in our code a "mult.vhd" submodule has also been developed and is declared and port mapped in submodules including "input_processing.vhd", "output_processing.vhd", "n_hidden.vhd", and "n_output.vhd". Finally, all these submodules are declared, and port mapped in the top module design "ann.vhd".

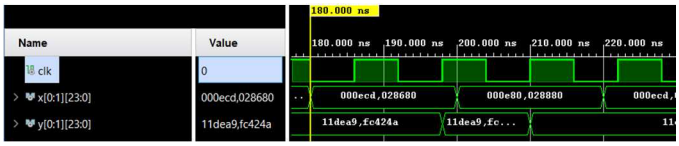
IV. Results and Conclusion

To verify the validity of the design a testbench named "ann_tb.vhd" has been written which reads the inputs with the help of the developed artificial neural network package. Consequently, the total number of 1199 input samples will generate associated 1198 samples as depicted in the Figure 6(a). In this figure the design latency for the ann4 with sfix_24_8 fixed-point model is 9 clocks which equals 108ns for this

Authorized licensed use limited to: Cyprus University of Technology. Downloaded on June 04,2025 at 12:04:18 UTC from IEEE Xplore. Restrictions apply.

scenario. The generated outputs for the ann8 with sfix_24_8 fixed-point model is also shown in the Figure 6(b).

(a)



(b)

Figure 6. (a) The generated output of ann4 sfix_24_8 fixed-point model and its relevant latency. (b) The generated output of ann8 sfix_24_8 fixed-point model and its relevant latency.

The latency remains the same due to the techniques used in the VHDL coding used that can be generalized to different number of hidden layer sizes and it is 9 clocks in both cases equaling 108 ns for the clock frequency of 83.3 MHz.

The Figure 7 depicts the power consumption and Worst Negative Slack (WNS) for the ann4 sfix_24_8 model for different clock frequencies. Similarly, the results of power consumption and Worst Negative Slack (WNS) based on the clock frequency has been depicted for ann4 sfix_32_16, ann8 sfix_24_8, and ann8 sfix_32_16 models in Figure 8, Figure 9, and Figure 10, respectively. These results verify the proportionate relation between the clock frequency and power consumption as well as the WNS and clock frequency in which the increase in the latter will drop the WNS and lead the design the be in verge of timing failure.

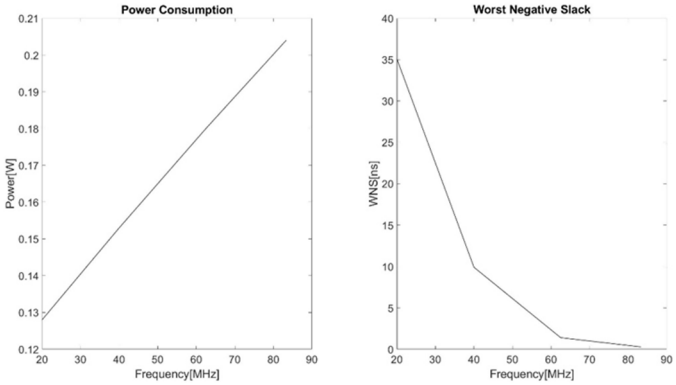


Figure 7. The power consumption and Worst Negative Slack with respect to the clock Frequency for ann4 sfix_24_8

As anticipated by increasing the number of bits the required resources in the FPGA increase thus the power consumption increases respectively while this increase leads to the reduction in the WNS, so a remarkable attention must be paid to choose the maximum achievable clock frequency to be suitable for the design to preserve the margin for the WNS. The clock frequency and the maximum achievable clock frequency for different architecture has been listed in the table II.

Table II concludes that the clock frequency drops by the increase in the number of bits as it increases the number of utilized resources in the FPGA thus makes the routing more complicated and the timing becomes the bottleneck. Therefore, it is observable that the clock frequency for the sfix_32_16 model of ann8 is less than that for ann4.

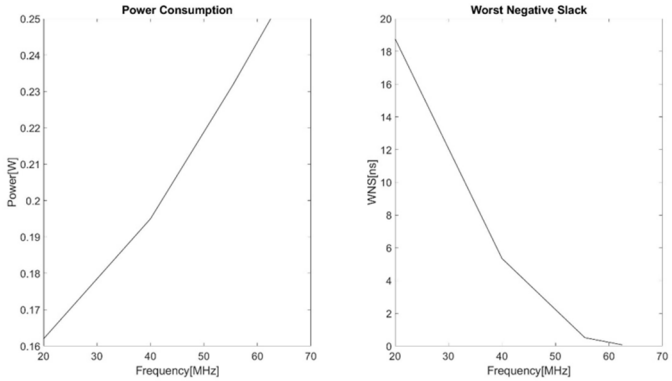


Figure 8. The power consumption and Worst Negative Slack with respect to the clock Frequency for ann4 sfix_32_16

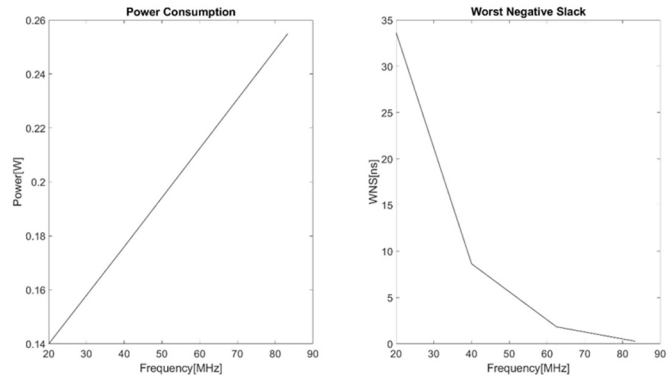


Figure 9. The power consumption and Worst Negative Slack with respect to the clock Frequency for ann8 sfix_24_8

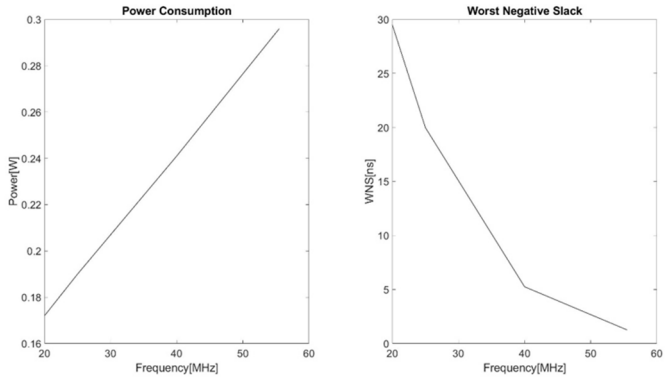


Figure 10. The power consumption and Worst Negative Slack with respect to the clock Frequency for ann8 sfix_32_16

The resource utilization of four different ANN architectures can also be compared. Initially, the implementation phase must be completed and then the utilization report will be ready to use.

Authorized licensed use limited to: Cyprus University of Technology. Downloaded on June 04,2025 at 12:04:18 UTC from IEEE Xplore. Restrictions apply.

The table III lists the number of Look-Up-Tables (LUTs), Flip Flops (FFs), DSP slices, and IOs used in each design.

As can be seen in the table III the change in the neural network topology by increasing the hidden layer size will contribute to the resources utilized rise dramatically while in each fixed topology the increase in the clock frequency has a negligible impact on the resources utilized. Figure 11 demonstrates the performance of ann4 sfix_24_8 RTL implementation. As it is transparent the values of the RTL implementation perfectly match with those of the sfix_24_8 Simulink model. This is also validated by the 0.0014 MSE error of output 1 before. Therefore, it is reasonable to choose sfix_24_8 model as the efficient one for the implementation as it provides remarkable performance in terms of accuracy, power consumption, clock frequency, and the resources utilization.

Table II. CLOCK FREQUENCY VS MAXIMUM ACHIEVABLE CLOCK FREQUENCY FOR DIFFERENT ANN ARCHITECTURES

Clock [MHz]	ann_4		ann_8	
	sfix_24_8	sfix_32_16	sfix_24_8	sfix_32_16
Clock Frequency	83.33	62.50	83.33	55.50
Maximum Clock Frequency	85.36	62.70	85.30	59.70

Table III. THE RESOURCE UTILIZATION FOR DIFFERENT ANN ARCHITECTURES

Resource Utilization		Freq	LUT	FF	DSP	IO
ann_4	sfix_24_8	20	990/53200	743/106400	35/220	97/200
		40	991/53200	743/106400	35/220	97/200
		62.5	991/53200	743/106400	35/220	97/200
		83.3	998/53200	743/106400	35/220	97/200
	sfix_32_16	20	3274/53200	964/106400	40/220	129/200
		40	3283/53200	964/106400	40/220	129/200
		55.5	3287/53200	964/106400	40/220	129/200
		62.5	3334/53200	964/106400	40/220	129/200
ann_8	sfix_24_8	20	1753/53200	1182/106400	61/220	97/200
		40	1754/53200	1182/106400	61/220	97/200
		62.5	1754/53200	1182/106400	61/220	97/200
		83.3	1762/53200	1182/106400	61/220	97/200
	sfix_32_16	20	5467/53200	1456/106400	72/220	129/200
		25	5467/53200	1456/106400	72/220	129/200
		40	5467/53200	1456/106400	72/220	129/200
		55.5	5469/53200	1456/106400	72/220	129/200

The implemented design on the Zedboard Evaluation FPGA for the ann4 sfix_24_8 model has been shown in the Figure 12. It is noted that the usage of resources and slices are mostly located in regions X0Y0 and X1Y0.

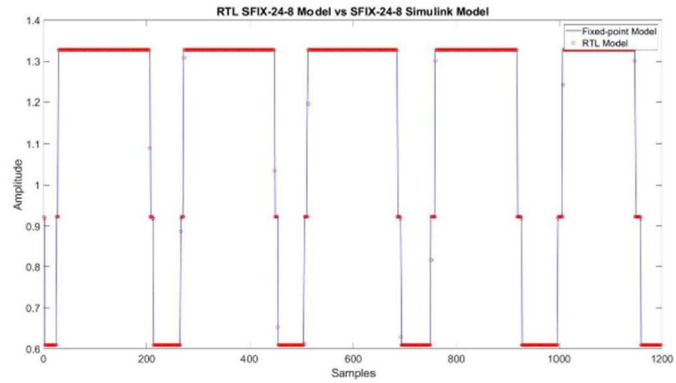


Figure 11. The accuracy of the ann4 sfix_24_8 RTL implementation vs sfix_24_8 Simulink model

The VHDL code is written such that the RTL design is fully capable of being generalized for different topologies of an artificial neural network. However, there are some design strategy tools both for synthesis and implementation offered by Vivado Design Suite that can be leveraged to enhance the performance of the design. Here, for each of the architectures I used three different implementation design strategies named "Performance_Explore", "Power_DefaultOpt", and "Area_Explore" to observe their impact on the design results. These enhancements only applied to one clock frequency for each of the architectures of the neural network. The result of this optimization is shown in Table IV. Note that these optimizations are based on Xilinx methods and before that the width of the weights and biases have also been optimized to use the efficient number of resources while obtaining the suitable accuracy. This has been previously discussed in Section II.

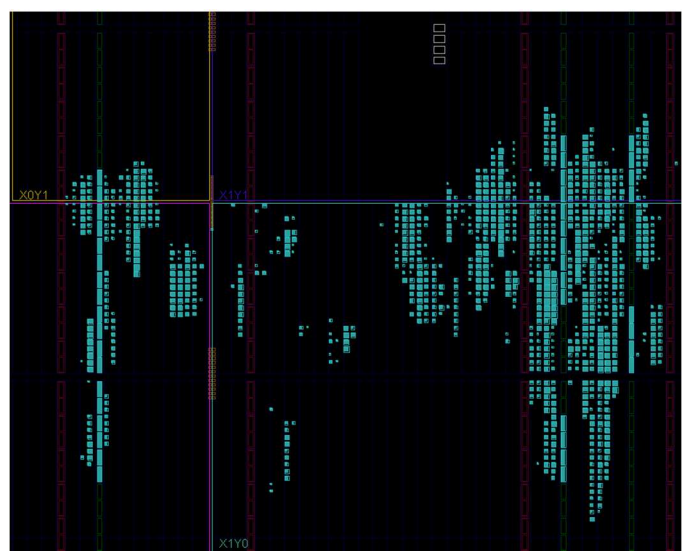


Figure 12. The implemented design of the ann4 sfix_24_8 on Zedboard Evaluation Board

Authorized licensed use limited to: Cyprus University of Technology. Downloaded on June 04,2025 at 12:04:18 UTC from IEEE Xplore. Restrictions apply.

Table IV. THE IMPACT OF IMPLEMENTATION DESIGN STRATEGIES ON PERFORMANCE

Architecture & Clock Freq [MHz]		Optimization	Power	WNS	LUT
ann_4	sfix_24_8 (83.3 MHz)	Performance	0.206	**0.285**	1006/53200
		Power	**0.203**	0.175	1005/53200
		Area	0.207	0.092	**991/53200**
		No Optimization	0.204	0.221	998/53200
	sfix_32_16 (55.5 MHz)	Performance	0.232	**0.922**	3287/53200
		Power	**0.230**	0.712	3287/53200
		Area	0.232	0.514	**3286/53200**
		No Optimization	0.232	0.514	3287/53200
ann_8	sfix_24_8 (83.3 MHz)	Performance	0.255	**0.403**	1777/53200
		Power	**0.252**	0.277	1762/53200
		Area	0.254	0.312	**1721/53200**
		No Optimization	0.255	0.277	1762/53200
	sfix_32_16 (55.5 MHz)	Performance	0.296	**1.382**	5469/53200
		Power	**0.295**	1.255	5504/53200
		Area	0.295	0.732	**5370/53200**
		No Optimization	0.296	1.255	5469/53200

The enhanced results have been highlighted in the table. As anticipated the Performance_Explore design strategy is intended to enhance the clock speed of the design and so the WNS improvement is the outcome of using this strategy leading to higher maximum achievable clock frequency. The Power_DefaultOpt method is used for the power consumption optimization and as can be seen in the table IV there is a slight improvement in the power leading to reduction compared to other conditions. Finally, the Area_Explore as its names indicates is used for the optimization of the area consequently the utilized resources. This is visible compared to other conditions in using the flip flop as an example. Last but not least, it is proved that each of these design strategies are superior to the normal design without optimization tool applied in their related area including speed, power, and area.

REFERENCES

[1] Philippe Dondon, Julien Carvalho, Rémi Gardere, Paul Lahalle, Georgi Tsenov, and Valeri Mladenov, "Implementation of a Feed-forward Artificial Neural Network in VHDL on FPGA," 12th Symposium on Neural Network Applications in Electrical Engineering (NEUREL), November 2014.

[2] R. Abdu-Aljabar, "Design and Implementation of Neural Network in FPGA", Journal of Engineering and Development, September 2018.

[3] Jesse Schmitz, Lei Zhang, "FPGA Hardware Implementation and Optimization for Neural Network based Chaotic System Design", The 9th International Symposium on Highly-Efficient Accelerators and Reconfigurable Technologies (HEART 2018), June 2018.

[4] Howard B. Demuth Mark Hudson Beale, Martin T. Hagan, "MATLAB Neural Network Toolbox User's Guide", 2017.

[5] Lei Zhang, "Implementation of Fixed-point Neuron Models with Threshold, Ramp and Sigmoid Activation Functions" 4th International Conference on Mechanics and Mechatronics Research (ICMMR 2017), June 2017.

[6] Kasem Khalil, Ashok Kumar, Magdy Bayoumi "Reconfigurable Hardware Design Approach for Economic Neural Network," IEEE Transactions on Circuits and Systems II: Express Briefs, pp. 5094–5098, July 2022.

[7] Leandro D. Medus, Taras Iakmychuk, Jose Vicente Frances-Villora, Manuel Bataller-Mompean, Alfredo Rosado-Munoz, " A Novel Systolic Parallel Hardware Architecture for the FPGA Acceleration of Feedforward Neural Networks", IEEE Access, June 2019.

[8] Xilinx, Vivado Design Suite Tutorial, "Zynq-7000 All Programmable SoC: Embedded Design Tutorial", UG1165, 2021.2.

[9] Xilinx, Vivado Design Suite Tutorial, "Model-Based Design Using Model Composer", UG1259, 2020.1.

Authorized licensed use limited to: Cyprus University of Technology. Downloaded on June 04,2025 at 12:04:18 UTC from IEEE Xplore. Restrictions apply.